ARTIFICIAL INTELLIGENCE

Assignment- 1

AI Voice-Controlled Home Assistant

Submitted By
MOHAMMED SAYIM
GROUP 12
S7 CSEA
Roll No:46

AI Voice-Controlled Home Assistant

1. Introduction

This report presents the design and implementation of an intelligent AI agent that controls home appliances through natural voice commands, featuring real-time speech recognition, natural neural voice responses, and a modern graphical user interface. The system enhances user convenience, accessibility and energy management by accurately interpreting spoken instructions, maintaining conversational context, and providing clear spoken feedback while operating under the uncertainty of real-world acoustic environments.

2. PEAS Description

Component	Description
Performance Measure	• Speech recognition & intent accuracy • Correct appliance action execution rate • Response latency and naturalness of spoken feedback • Minimization of false activations • User satisfaction and energy savings
Environment	Multi-room smart home with IoT appliances (lights, fans, ACs, TVs, plugs), ambient noise, multiple users, varying acoustic conditions and continuous device state changes. Interaction occurs through microphone and speakers.
Actuators	• Text-to-speech engine (neural female voice) for spoken replies • Appliance controllers (simulated / IoT ready) • Graphical UI updates (device status, conversation history)
Sensors	• Microphone for continuous voice capture • Automatic Speech Recognition (Google ASR) • Appliance state sensors (on/off, temperature, brightness, volume)

3. Environment Classification

Property	Class	Justification
Observability	Partially Observable	Background noise, accents and incomplete utterances prevent perfect perception of intent.
Determinism	Stochastic	ASR errors and unpredictable user phrasing make outcomes probabilistic.
Episode	Sequential	Commands depend on prior context (“turn it off”). Dialogue state is maintained.
Dynamism	Dynamic	Device states, noise levels and user presence change independently over time.
Continuity	Discrete actions	Control actions are discrete; continuous signals exist at the audio sensor level.
Agents	Single-Agent	One assistant agent interacts with multiple users and devices.

4. Rationality Measure

A rational agent selects actions that maximize expected performance. The Voice Home Assistant achieves rationality by: (1) converting speech to text via ASR, (2) extracting intent and entities with a rule-based NLU layer, (3) resolving ambiguous references using dialogue context (last device and room), (4) applying safety checks and device constraints before acting, and (5) giving clear spoken confirmation so the user can correct errors. When confidence is low the agent asks for clarification instead of risking an incorrect action. This policy maximizes successful executions while minimizing unsafe or unwanted changes.

5. Innovation Points

Three innovative features are implemented beyond basic voice control systems: (1) a modern graphical UI that shows live device states and conversation history for transparency and usability; (2) high-quality neural text-to-speech (edge-tts with a natural female voice) that replaces robotic SAPI voices; (3) context-aware sequential reasoning that correctly handles incomplete commands such as “turn it off” or “increase the AC”. These features improve naturalness, user trust and practical usability of the smart-home assistant.

6. System Architecture Diagram

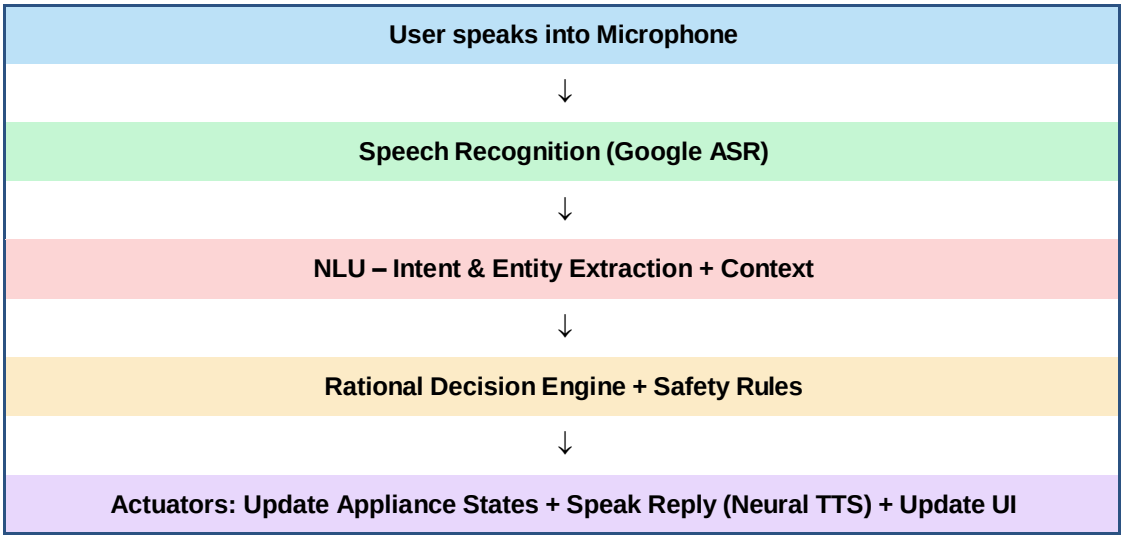


Figure 1: Architecture – Sensors → Reasoning → Actuators (Voice + UI)

7. Conclusion

The implemented AI Voice-Controlled Home Assistant is a complete rational agent that listens through a microphone, interprets natural language commands, maintains context, controls appliances and replies with a natural female neural voice, all inside a modern graphical interface. The design successfully handles partial observability and stochastic speech recognition while offering a practical and user-friendly foundation for real smart-home applications.

8. Simulation

A fully working Python application was developed. It uses the system microphone for continuous listening, Google Speech Recognition for ASR, a rule-based NLU engine with context, and high-quality neural TTS (edge-tts) for spoken replies. A CustomTkinter graphical interface displays conversation history, live device status and listening state. The same logic also works in pure console mode. Students can run the program, issue voice commands and capture screenshots of both the UI and the terminal for submission.

9. Full Source Code

```
import os
import sys
import threading
import tempfile
import asyncio
import speech_recognition as sr
from typing import Dict, Optional
from dataclasses import dataclass
from enum import Enum

# UI
import customtkinter as ctk
from tkinter import messagebox

# Better voice
try:
    import edge_tts
    import pygame
    HAS_EDGE = True
except ImportError:
    HAS_EDGE = False

# Fallback TTS
try:
    import win32com.client
    HAS_SAPI = True
except ImportError:
    HAS_SAPI = False


# ============================================================
# CONFIG
# ============================================================
VOICE = "en-US-JennyNeural"   # Natural female voice (very good)
# Other good options: en-US-AriaNeural, en-GB-SoniaNeural, en-US-MichelleNeural

ctk.set_appearance_mode("Dark")
```

```python
ctk.set_default_color_theme("blue")


# ============================================================
# SPEAKING
# ============================================================
def speak_edge(text: str):
    """High quality neural voice using edge-tts"""
    async def _speak():
        communicate = edge_tts.Communicate(text, VOICE)
        temp_file = os.path.join(tempfile.gettempdir(), "assistant_reply.mp3")
        await communicate.save(temp_file)
        pygame.mixer.init()
        pygame.mixer.music.load(temp_file)
        pygame.mixer.music.play()
        while pygame.mixer.music.get_busy():
            await asyncio.sleep(0.1)
        pygame.mixer.quit()
        try:
            os.remove(temp_file)
        except:
            pass

    try:
        asyncio.run(_speak())
    except Exception as e:
        print("edge-tts error:", e)
        speak_fallback(text)


def speak_fallback(text: str):
    """Windows SAPI fallback"""
    if HAS_SAPI:
        try:
            speaker = win32com.client.Dispatch("SAPI.SpVoice")
            # Try to select a female voice
            voices = speaker.GetVoices()
            for i in range(voices.Count):
                v = voices.Item(i)
```

```
                if "zira" in v.GetDescription().lower() or "female" in v.GetDescription().lower():
                    speaker.Voice = v
                    break
            speaker.Speak(text)
            return
        except:
            pass
    print(f"[SPEAK] {text}")


def speak(text: str, ui_callback=None):
    if ui_callback:
        ui_callback(text)
    if HAS_EDGE:
        speak_edge(text)
    else:
        speak_fallback(text)


# ============================================================
# HOME MODEL
# ============================================================
class ApplianceType(Enum):
    LIGHT = "light"
    FAN = "fan"
    AC = "ac"
    TV = "tv"
    PLUG = "plug"


@dataclass
class Appliance:
    name: str
    room: str
    type: ApplianceType
    is_on: bool = False
    value: Optional[int] = None
```

```python
    def nice(self):
        return self.name.replace("_", " ").title()


class Home:
    def __init__(self):
        self.devices: Dict[str, Appliance] = {}
        data = [
            ("living_room_light", "Living Room", ApplianceType.LIGHT),
            ("bedroom_light", "Bedroom", ApplianceType.LIGHT),
            ("kitchen_light", "Kitchen", ApplianceType.LIGHT),
            ("living_room_fan", "Living Room", ApplianceType.FAN),
            ("bedroom_fan", "Bedroom", ApplianceType.FAN),
            ("living_room_ac", "Living Room", ApplianceType.AC),
            ("bedroom_ac", "Bedroom", ApplianceType.AC),
            ("living_room_tv", "Living Room", ApplianceType.TV),
            ("smart_plug", "Kitchen", ApplianceType.PLUG),
        ]
        for n, r, t in data:
            a = Appliance(n, r, t)
            if t == ApplianceType.AC:
                a.value = 24
            elif t == ApplianceType.LIGHT:
                a.value = 70
            elif t == ApplianceType.TV:
                a.value = 30
            self.devices[n] = a

    def status_text(self):
        parts = []
        for a in self.devices.values():
            state = "ON" if a.is_on else "OFF"
            if a.value is not None and a.is_on:
                parts.append(f"{a.nice()} is {state} ({a.value})")
            else:
                parts.append(f"{a.nice()} is {state}")
        return "Current status: " + ". ".join(parts)

# ============================================================
# AGENT
# ============================================================
class Agent:
    def __init__(self, home: Home):
        self.home = home
        self.last_device = None
        self.last_room = None

    def handle(self, text: str) -> str:
        cmd = text.lower().strip()
        print(f"[DEBUG] Raw command: '{cmd}'")

        if any(w in cmd for w in ["exit", "quit", "goodbye", "bye", "stop"]):
            return "EXIT"

        if any(w in cmd for w in ["status", "what is the status", "report", "list devices", "device status"]):
            return self.home.status_text()

        # ----- Detect ACTION -----
        action = None
        if any(w in cmd for w in ["turn on", "switch on", "switch on the", "on the", "start the"]):
            action = "on"
        elif any(w in cmd for w in ["turn off", "switch off", "switch off the", "off the", "stop the"]):
            action = "off"
        elif any(w in cmd for w in ["set", "change to", "make it"]):
            action = "set"
        elif any(w in cmd for w in ["increase", "raise", "up", "higher", "brighter", "louder"]):
            action = "increase"
        elif any(w in cmd for w in ["decrease", "lower", "down", "dim", "quieter", "reduce"]):
            action = "decrease"

        # ----- Detect ROOM (more flexible) -----
        room = None
        if "living room" in cmd or "livingroom" in cmd or "hall" in cmd:
            room = "Living Room"
        elif "bed room" in cmd or "bedroom" in cmd or "bed" in cmd:
            room = "Bedroom"
```

```
        for d in self.home.devices.values():
            if d.type == atype and d.room == self.last_room:
                target = d
                break

    # Priority 3: incomplete command ("turn it off") → use last device
    elif action in ["on", "off", "increase", "decrease", "set"] and self.last_device:
        target = self.home.devices.get(self.last_device)

    if target is None:
        return "I could not identify the device. Please clearly say the room and the device, for example: turn on the bedroom light."

    print(f"[DEBUG] Matched device: {target.name}")

    # ----- EXECUTE ACTION -----
    if action == "on":
        if target.is_on:
            reply = f"The {target.nice()} is already on."
        else:
            target.is_on = True
            reply = f"Okay, turning on the {target.nice()}."
    elif action == "off":
        if not target.is_on:
            reply = f"The {target.nice()} is already off."
        else:
            target.is_on = False
            reply = f"Okay, turning off the {target.nice()}."
    elif action == "set" and value is not None:
        target.value = value
        target.is_on = True
        reply = f"Setting {target.nice()} to {value}."
    elif action == "increase" and target.value is not None:
        maxv = 30 if target.type == ApplianceType.AC else 100
        target.value = min(maxv, target.value + 5)
        target.is_on = True
        reply = f"Increased {target.nice()} to {target.value}."
    elif action == "decrease" and target.value is not None:
```

```
        title = ctk.CTkLabel(left, text="AI Voice Home Assistant",
                             font=ctk.CTkFont(size=22, weight="bold"))
        title.pack(pady=(15, 5))

        subtitle = ctk.CTkLabel(left, text="Speak naturally • I listen & respond",
                                font=ctk.CTkFont(size=13), text_color="gray")
        subtitle.pack(pady=(0, 10))

        self.chat_box = ctk.CTkTextbox(left, font=ctk.CTkFont(size=14),
                                       wrap="word", state="disabled",
                                       corner_radius=10)
        self.chat_box.pack(fill="both", expand=True, padx=10, pady=5)

        # Status indicator
        self.status_label = ctk.CTkLabel(left, text="● Ready",
                                         font=ctk.CTkFont(size=13),
                                         text_color="#4ade80")
        self.status_label.pack(pady=5)

        # Buttons
        btn_frame = ctk.CTkFrame(left, fg_color="transparent")
        btn_frame.pack(pady=10)

        self.listen_btn = ctk.CTkButton(btn_frame, text="🎤  Start Listening",
                                        width=180, height=40,
                                        font=ctk.CTkFont(size=14, weight="bold"),
                                        command=self.toggle_listen)
        self.listen_btn.pack(side="left", padx=6)

        self.status_btn = ctk.CTkButton(btn_frame, text="📄 Status",
                                        width=110, height=40,
                                        command=self.show_status)
        self.status_btn.pack(side="left", padx=6)

        # Right panel - Devices
        right = ctk.CTkFrame(self, width=280, corner_radius=12)
```

```
    right.pack(side="right", fill="y", padx=(0, 12), pady=12)
    right.pack_propagate(False)

    ctk.CTkLabel(right, text="Home Devices",
                 font=ctk.CTkFont(size=16, weight="bold")).pack(pady=(15, 10))

    self.device_frame = ctk.CTkScrollableFrame(right, corner_radius=8)
    self.device_frame.pack(fill="both", expand=True, padx=10, pady=5)

    self.device_labels = {}
    for name, app in self.home.devices.items():
        f = ctk.CTkFrame(self.device_frame, corner_radius=8)
        f.pack(fill="x", pady=4, padx=4)
        lbl = ctk.CTkLabel(f, text=self._device_text(app),
                           font=ctk.CTkFont(size=12), anchor="w")
        lbl.pack(fill="x", padx=8, pady=6)
        self.device_labels[name] = lbl

    ctk.CTkLabel(right, text="Group 12 Assignment",
                 font=ctk.CTkFont(size=11), text_color="gray").pack(pady=10)

def _device_text(self, app: Appliance) -> str:
    icon = "🟢" if app.is_on else "⚫"
    state = "ON" if app.is_on else "OFF"
    if app.value is not None and app.is_on:
        return f"{icon}  {app.nice()}\n     {state}  •  {app.value}"
    return f"{icon}  {app.nice()}\n     {state}"

def refresh_devices(self):
    for name, app in self.home.devices.items():
        self.device_labels[name].configure(text=self._device_text(app))

def add_chat(self, who: str, text: str):
    self.chat_box.configure(state="normal")
    if who == "You":
        self.chat_box.insert("end", f"You:  {text}\n\n")
    else:
        self.chat_box.insert("end", f"Assistant:  {text}\n\n")
    self.chat_box.see("end")
    self.chat_box.configure(state="disabled")

def set_status(self, text: str, color: str = "#4ade80"):
    self.status_label.configure(text=text, text_color=color)

def welcome(self):
    msg = "Hello! I am your AI home assistant. How can I help you today?"
    self.add_chat("Assistant", msg)
    threading.Thread(target=lambda: speak(msg), daemon=True).start()

def show_status(self):
    reply = self.home.status_te
    self.add_chat("Assistant", reply)
    threading.Thread(target=lambda: speak(reply), daemon=True).start()
    self.refresh_devices()

def toggle_listen(self):
    if not self.listening:
        self.listening = True
        self.stop_flag = False
        self.listen_btn.configure(text="⏹  Stop Listening", fg_color="#dc2626")
        self.set_status("● Listening...", "#facc15")
        threading.Thread(target=self.listen_loop, daemon=True).start()
    else:
        self.stop_flag = True
        self.listening = False
        self.listen_btn.configure(text="🎤  Start Listening",
                                  fg_color=["#3B8ED0", "#1F6AA5"])
        self.set_status("● Ready", "#4ade80")

def listen_loop(self):
    mic = sr.Microphone()
    while self.listening and not self.stop_flag:
        try:
```

(variable) reply: LiteralString

```
        try:
            with mic as source:
                self.recognizer.adjust_for_ambient_noise(source, duration=0.3)
                self.set_status("● Listening... Speak now", "#facc15")
                audio = self.recognizer.listen(source, timeout=5, phrase_time_limit=6)

            self.set_status("● Recognizing...", "#60a5fa")
            text = self.recognizer.recognize_google(audio)
            self.after(0, lambda t=text: self.process_command(t))

        except sr.WaitTimeoutError:
            continue
        except sr.UnknownValueError:
            self.after(0, lambda: self.set_status("● Could not understand", "#f87171"))
            continue
        except Exception as e:
            print("Listen error:", e)
            self.after(0, lambda: self.set_status("● Error", "#f87171"))
            break

    self.after(0, lambda: self.set_status("● Ready", "#4ade80"))

def process_command(self, text: str):
    self.add_chat("You", text)
    reply = self.agent.handle(text)

    if reply == "EXIT":
        self.add_chat("Assistant", "Goodbye! Have a nice day.")
        threading.Thread(target=lambda: speak("Goodbye! Have a nice day."), daemon=True).start()
        self.stop_flag = True
        self.listening = False
        self.listen_btn.configure(text="🎤  Start Listening",
                                  fg_color=["#3B8ED0", "#1F6AA5"])
        return

    self.add_chat("Assistant", reply)
        self.add_chat("Assistant", reply)
        self.refresh_devices()
        threading.Thread(target=lambda: speak(reply), daemon=True).start()
        self.set_status("● Ready", "#4ade80")

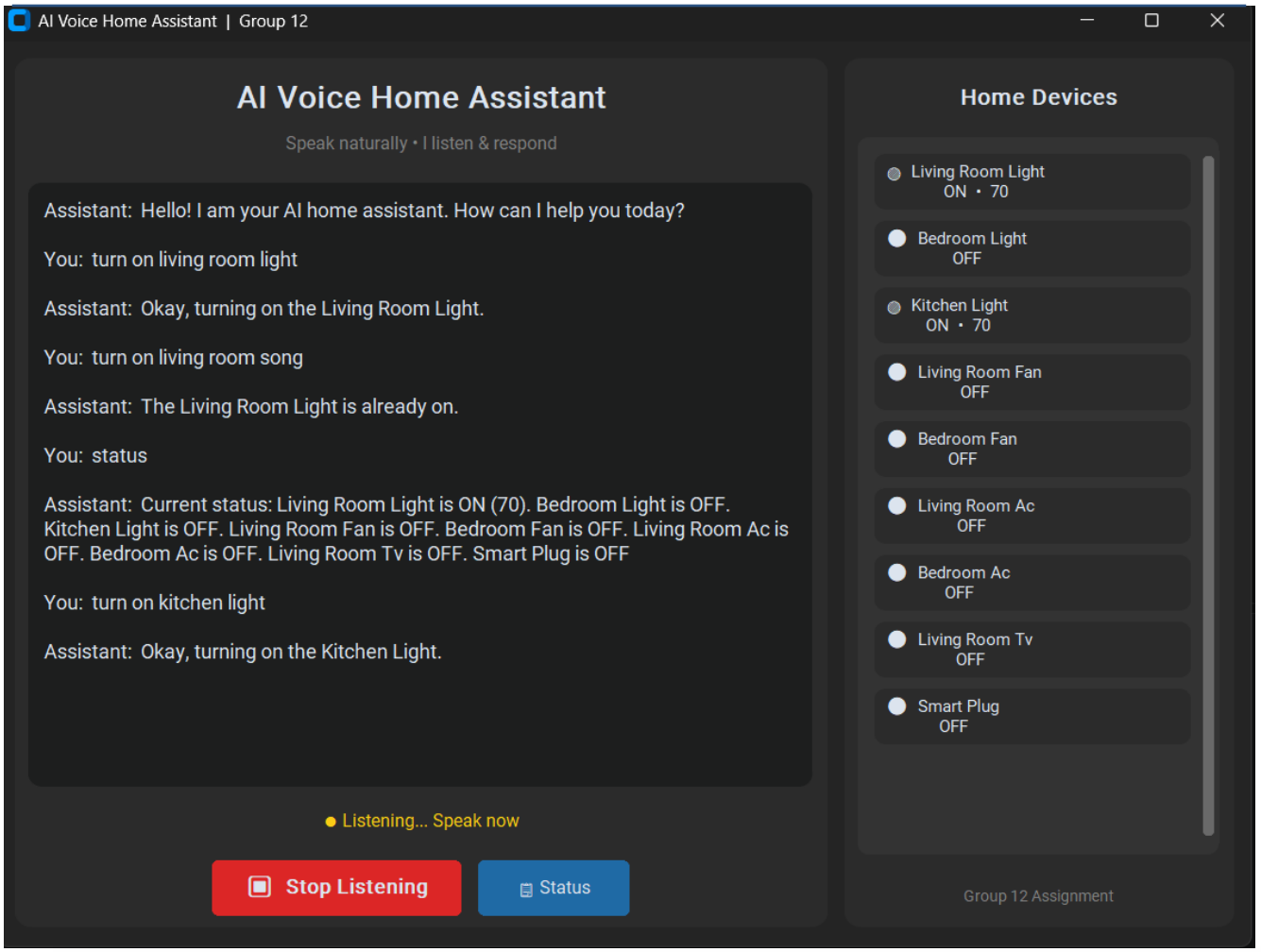

# ============================================================
# MAIN
# ============================================================
if __name__ == "__main__":
    # Check critical packages
    missing = []
    try:
        import customtkinter
    except ImportError:
        missing.append("customtkinter")
    try:
        import speech_recognition
    except ImportError:
        missing.append("SpeechRecognition")

    if missing:
        print("Please install missing packages:")
        print("  pip install " + " ".join(missing))
        print("  pip install edge-tts pygame pywin32 pyaudio")
        sys.exit(1)

    app = App()
    app.mainloop()
```

10. OUTPUT

When the application starts, the assistant greets the user with a natural female voice and the UI appears.




```
PS C:\Users\jeesm\Downloads> python voice_assistant_ui.py
pygame 2.6.1 (SDL 2.28.4, Python 3.11.6)
Hello from the pygame community. https://www.pygame.org/contribute.html
[DEBUG] Raw command: 'turn on living room light'
[DEBUG] Detected → action=on, room=Living Room, type=ApplianceType.LIGHT, value=None
[DEBUG] Matched device: living_room_light
[DEBUG] Raw command: 'turn on living room song'
[DEBUG] Detected → action=on, room=Living Room, type=None, value=None
[DEBUG] Matched device: living_room_light
[DEBUG] Raw command: 'status'
[DEBUG] Raw command: 'turn on kitchen light'
[DEBUG] Detected → action=on, room=Kitchen, type=ApplianceType.LIGHT, value=None
[DEBUG] Matched device: kitchen_light
edge-tts error: [Errno 13] Permission denied: 'C:\\Users\\jeesm\\AppData\\Local\\Temp\\assistant_reply.mp3'
[SPEAK] Okay, turning on the Kitchen Light.
PS C:\Users\jeesm\Downloads>
```